BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH

BÁO CÁO PROJECT CÁ NHÂN
MÔN HỆ ĐIỀU HÀNH THỜI GIAN THỰC (RTOS)

ĐỀ TÀI:

ỨNG DỤNG FREERTOS TRÊN ESP32: 8 TASK ĐỒNG BỘ BẰNG SEMAPHORE, MUTEX, QUEUE, EVENT GROUP VÀ TASK NOTIFICATION

GVHD:	Huỳnh Hoàng Hà
Sinh viên:	Đặng Đình Mạnh
MSSV:	24119055
Học kỳ:	HK1 2026–2027, đợt 2

TP. Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

1. Giới thiệu và yêu cầu đề.....3
2. Tìm hiểu FreeRTOS.....3
2.1. Kiến trúc.....3
2.2. Đặc điểm.....4
2.3. Nguyên lý: trạng thái task và bộ lập lịch.....4
2.4. Các thành phần.....5
2.5. Chiến lược thiết kế đã áp dụng.....5
3. Thiết kế hệ thống.....6
3.1. Phần cứng.....6
3.2. Phần mềm.....7
4. Phân tích chương trình.....7
4.1. Tạo đối tượng kernel và tạo task.....7
4.2. Task 1: LED1 nháy T = 200 ms với vTaskDelayUntil.....9
4.3. Task 2: đọc cảm biến, lưu biến toàn cục, bảo vệ bằng mutex.....9
4.4. Task 3: BTN1 đảo LED2, vòng lặp quét có chống dội.....10
4.5. Task 4 và Task 5: đồng bộ bằng binary semaphore.....10
4.6. Task 6: hiển thị OLED mỗi 500 ms.....12
4.7. Các kỹ thuật mở rộng.....12
5. Kiểm thử và kết quả.....16
5.1. Build.....16
5.2. Chạy giả lập trên QEMU.....16
5.3. Kiểm thử trên kit.....18
6. Kết luận.....18
Tài liệu tham khảo.....19

1. Giới thiệu và yêu cầu đề

Project dùng FreeRTOS (có sẵn trong ESP-IDF, gọi qua Arduino framework) trên **ESP32-S3 DevKitC-1** để chạy song song nhiều task điều khiển LED, nút nhấn, cảm biến ánh sáng và màn hình OLED. Chương trình làm đủ 6 task của đề, thêm 2 task và 1 software timer để áp dụng các kỹ thuật FreeRTOS khác. Mỗi kỹ thuật trong báo cáo đều kèm đoạn code thật (tên file, số dòng) làm minh chứng.

Môi trường: VS Code + PlatformIO, platform pioarduino 55.03.30 (Arduino-ESP32 3.3.0, ESP-IDF 5.5, FreeRTOS **V10.5.1** bản SMP 2 lõi, tick 1000 Hz). Mã nguồn: thư mục `01_Personal_FreeRTOS_ESP32/`.

Bảng 1. Yêu cầu của đề và vị trí hiện thực

Task	Yêu cầu	Hàm	Kỹ thuật chính
1	LED1 nháy với T = 200 ms	`task1_led1()`	vTaskDelayUntil
2	Đọc cảm biến, lưu biến toàn cục, 500 ms/lần	`task2_sensor()`	mutex, vTaskDelayUntil
3	Button1 đảo trạng thái LED2 (vòng lặp)	`task3_btn1()`	polling + chống dội
4	Nhấn Button2 để cấp binary semaphore	`task4_btn2()`	xSemaphoreGive
5	Đợi semaphore, có thì bật LED3 3 s rồi tắt, đợi tiếp	`task5_led3()`	xSemaphoreTake
6	OLED: trạng thái LED2, cảm biến, LED3 đếm ngược, 500 ms/lần	`task6_oled()`	mutex, critical section
7 (thêm)	Điều khiển hệ thống bằng lệnh Serial, in thống kê định kỳ	`task7_control()`	queue set, suspend/resume/prio
8 (thêm)	Báo tối khi ánh sáng dưới ngưỡng	`task8_alarm()`	task notification
timer	Mỗi 10 s báo Task 7 in thống kê	`stat_timer_cb()`	software timer + semaphore

2. Tìm hiểu FreeRTOS

2.1. Kiến trúc

FreeRTOS là nhân hệ điều hành thời gian thực mã nguồn mở (giấy phép MIT), viết bằng C, chia làm hai phần:

- **Phần chung** (không phụ thuộc phần cứng): `tasks.c` (bộ lập lịch, danh sách task), `queue.c` (queue, semaphore, mutex đều xây trên queue), `timers.c`, `event_groups.c`, `stream_buffer.c`.
- **Phần port**: chuyển ngữ cảnh, ngắt tick, critical section cho từng CPU. Trên ESP32 là port Xtensa của Espressif, bản **SMP**: một bộ lập lịch phục vụ 2 lõi, task có thể ghim vào 1 lõi.
- **Cấu hình** qua `FreeRTOSConfig.h` (ESP-IDF sinh từ menuconfig): tần số tick, số mức ưu tiên (25 trên ESP32), kích thước stack của task hệ thống, bật/tắt từng tính năng.

Ứng dụng không có vòng `while(1)` duy nhất như lập trình siêu vòng lặp (super-loop). Thay vào đó mỗi chức năng là một task độc lập có stack riêng; nhân quyết định task nào chạy tại mỗi thời điểm.

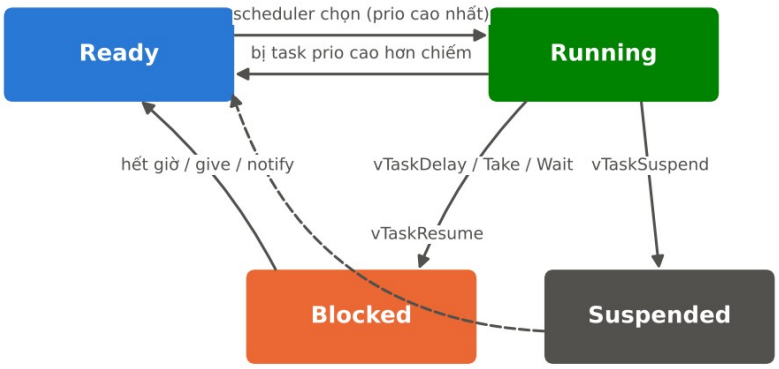
2.2. Đặc điểm

- **Lập lịch ưu tiên, có chiếm quyền (preemptive)**: luôn chạy task Ready có ưu tiên cao nhất. Task ưu tiên cao vừa sẵn sàng sẽ chiếm CPU ngay, không đợi task đang chạy nhả.
- **Xoay vòng (round-robin) theo tick** giữa các task cùng mức ưu tiên (time slicing).
- **Tất định**: thời gian chuyển task và API không phụ thuộc số task; đây là điều kiện để gọi là "thời gian thực".
- **Nhỏ gọn**: nhân vài KB flash; mọi đối tượng có thể cấp phát động (heap) hoặc tĩnh.
- **Thời gian tính theo tick**: mọi hàm chờ nhận số tick; macro `pdMS_TO_TICKS()` đổi ms sang tick. Ở project này 1 tick = 1 ms.

2.3. Nguyên lý: trạng thái task và bộ lập lịch

Mỗi task luôn ở một trong bốn trạng thái (Hình 3). Điểm cốt lõi: task **Blocked** (đang chờ thời gian hoặc sự kiện) **không tốn CPU**. Vì vậy task đúng chuẩn RTOS phải dành phần lớn thời gian ở trạng thái Blocked, chỉ thức dậy khi có việc. Nếu một task ưu tiên cao chạy vòng lặp không bao giờ block, mọi task ưu tiên thấp hơn trên cùng lõi sẽ **bị đói (starvation)**.

Hình 3. Các trạng thái task trong FreeRTOS



Hình 3. Chuyển trạng thái task

Ngắt tick (1 kHz) đánh thức bộ lập lịch: tăng bộ đếm tick, chuyển các task hết thời gian chờ từ Blocked sang Ready, rồi chọn task ưu tiên cao nhất. Khi không có task nào Ready, **idle task** (ưu tiên 0) chạy; trên ESP32 idle task còn nuôi watchdog và giải phóng bộ nhớ task đã xoá.

2.4. Các thành phần

Bảng 2. Các đối tượng FreeRTOS và dùng ở đâu trong project

Thành phần	Bản chất / dùng khi nào	Trong project
Task	Hàm `void f(void*)` vòng lặp vô hạn, có stack và priority riêng	8 task (Bảng 4)
Queue	Hàng đợi FIFO, copy dữ liệu theo giá trị, có chờ khi rỗng/đầy. Truyền dữ liệu giữa task	`g_cmdQueue`: dòng lệnh UART → Task 7
Queue set	Gom nhiều queue/semaphore; một task chờ **cùng lúc** tất cả, biết sự kiện đến từ đối tượng nào	`g_ctrlSet` = {`g_cmdQueue`, `g_statSem`}
Binary semaphore	Cờ 0/1, dùng để **đồng bộ** (báo sự kiện). Ai cũng give được	`g_btn2Sem`: Task 4 → Task 5; `g_statSem`: timer → Task 7
Counting semaphore	Đếm nhiều sự kiện / quản lý N tài nguyên	không dùng (giải thích ở 4.5)
Mutex	Semaphore đặc biệt để **loại trừ tương hỗ**, có kế thừa ưu tiên, chỉ task giữ mới được nhả	`g_lightMutex` bảo vệ `g_light`
Recursive mutex	Mutex mà task đang giữ được lấy lại thêm lần nữa (đếm số lần), nhả đủ số lần mới mở	`g_serialMutex` bảo vệ Serial
Event group	Tập bit cờ; một task chờ được nhiều điều kiện (AND/OR); có `xEventGroupSync` làm điểm hẹn	`g_events`: điểm hẹn khởi động, cờ cảm biến
Task notification	Mỗi task có sẵn 1 giá trị 32 bit; gửi thẳng tới task, nhanh và nhẹ hơn semaphore/queue	Task 2 → Task 8
Software timer	Callback chạy trong timer daemon task sau một khoảng thời gian, một lần hoặc lặp	`g_statTimer` 10 s
Critical section	Tắt ngắt + spinlock (2 lõi) cho đoạn rất ngắn	`g_led3Lock`

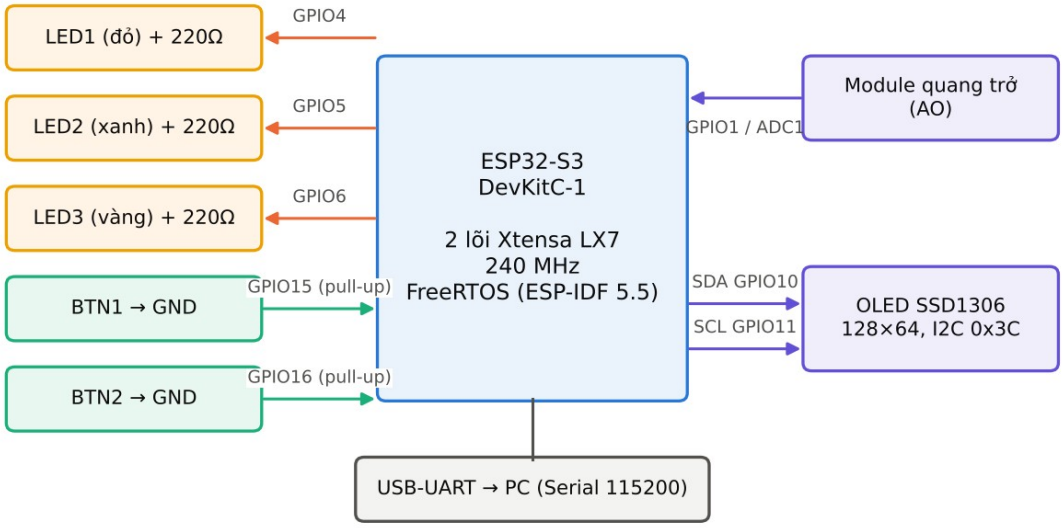
2.5. Chiến lược thiết kế đã áp dụng

- **Gán ưu tiên theo hạn chót**: việc cần đáp ứng nhanh (LED3 phản hồi semaphore, nút nhấn) ưu tiên cao; việc chậm, không gấp (vẽ OLED, in log) ưu tiên thấp nhất.
- **Chia lõi**: core 1 cho I/O (LED, nút), core 0 cho tác vụ nền (ADC, OLED I2C, Serial, cảnh báo). Việc vẽ OLED tốn ~25 ms không ảnh hưởng nhịp LED.
- **Mỗi tài nguyên dùng chung có cách bảo vệ riêng**: bus I2C chỉ Task 6 dùng nên không cần khoá; Serial nhiều task cùng in nên có recursive mutex.
- **Hướng sự kiện, không quét**: Task 7 nằm chờ queue set, chỉ thức dậy khi có lệnh hoặc khi timer báo.
- **Giữ khoá ngắn**: copy dữ liệu dưới khoá rồi nhả ngay, xử lý chậm làm bên ngoài.
- **Không chờ vô hạn khi lấy mutex**: dùng timeout, hết giờ thì bỏ lượt, hệ thống không treo.

3. Thiết kế hệ thống

3.1. Phần cứng

Hình 1. Sơ đồ kết nối phần cứng



Hình 1. Sơ đồ kết nối

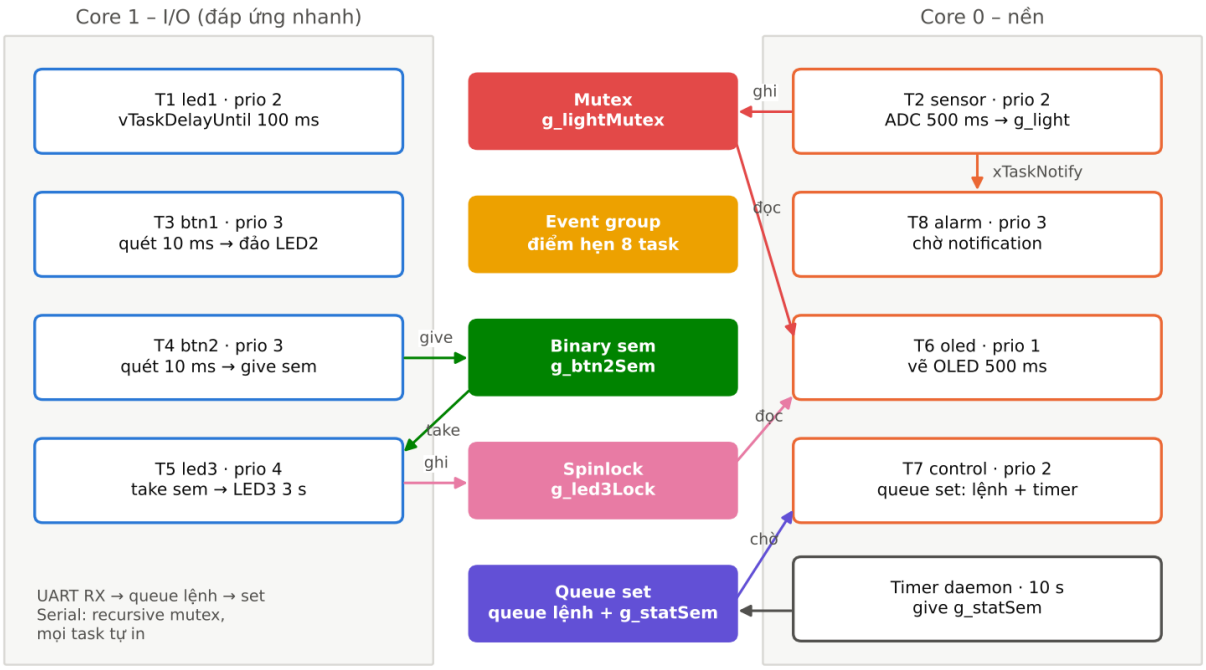
Bảng 3. Bảng chân (định nghĩa trong include/config.h)

Linh kiện	Chân ESP32-S3	Ghi chú
LED1, LED2, LED3 (+220 Ω)	GPIO4, 5, 6	mức cao = sáng
BTN1, BTN2 (nối GND)	GPIO15, 16	`INPUT_PULLUP`: nhả = 1, nhấn = 0
Module cảm biến (AO)	GPIO1 (ADC1_CH0)	dùng ADC1 (ADC2 xung đột WiFi); suy hao 11 dB đo tới ~3,1 V
OLED SSD1306 128×64	SDA 10, SCL 11	I2C 400 kHz, 0x3C (tự thử 0x3D). Không dùng GPIO8 vì trên kit bị kéo xuống GND

Kit và mạch testboard dùng chung với nhóm (được GV cho phép), nên bảng chân giống bài của bạn cùng nhóm; chương trình là bản riêng. Các chân tránh chân strapping (0, 3, 45, 46), chân USB (19, 20), chân flash/PSRAM (26–37) và UART0 (43, 44). Có sẵn bảng chân thứ hai cho ESP32 thường (env `esp32dev`).

3.2. Phần mềm

Hình 2. Kiến trúc phần mềm: 8 task, 2 lõi và đối tượng đồng bộ



Hình 2. Kiến trúc phần mềm

Bảng 4. Các task

Task	Core	Prio	Stack (B)	Chu kỳ / sự kiện	Lý do chọn ưu tiên
T1 led1	1	2	3072	100 ms (T = 200 ms)	nhịp đều, việc rất ngắn
T2 sensor	0	2	3072	500 ms	định kỳ, không gấp
T3 btn1	1	3	3072	quét 10 ms	nút phải phản hồi nhanh
T4 btn2	1	3	3072	quét 10 ms	như T3
T5 led3	1	4	3072	chờ semaphore	cao nhất: LED3 sáng ngay khi nhấn
T6 oled	0	1	4096	500 ms	chậm (I2C), không gấp
T7 control	0	2	4096	chờ queue set	lệnh người dùng, phản hồi nhanh hơn OLED
T8 alarm	0	3	3072	chờ notification	cảnh báo cần báo ngay

Mã nguồn tách theo chức năng: `main.cpp` (tạo đối tượng, tạo task), `task_io.cpp` (T1, T3, T4, T5), `task_sensor.cpp` (T2, T8), `task_display.cpp` (T6), `task_control.cpp` (T7, log, timer). Mọi đối tượng dùng chung khai báo trong `include/app.h`, có ghi rõ task nào ghi, task nào đọc.

4. Phân tích chương trình

4.1. Tạo đối tượng kernel và tạo task

Đối tượng kernel phải được tạo **trước** task dùng nó, và kiểm tra NULL vì có thể thiếu heap:

src/main.cpp (dòng 95–114)

```
 95    // Tạo đối tượng kernel TRƯỚC khi tạo task dùng chúng
 96    g_btn2Sem    = xSemaphoreCreateBinary();                          // [K5] tạo ra
ở trạng thái rỗng (0)
 97    g_lightMutex = xSemaphoreCreateMutex();                           // [K6] có
priority inheritance
 98    g_serialMutex = xSemaphoreCreateRecursiveMutex();                // [K6b]
 99    g_cmdQueue   = xQueueCreate(CMD_QUEUE_LEN, sizeof(CmdLine_t));    // [K8] copy
theo giá trị
100    g_statSem    = xSemaphoreCreateBinary();
101    // Queue set phải đủ chỗ cho tổng số phần tử của các thành viên: 4 dòng lệnh + 1
token semaphore.
102    // Thành viên phải đang RỖNG khi thêm vào set.
103    g_ctrlSet    = xQueueCreateSet(CMD_QUEUE_LEN + 1);
104    if (g_ctrlSet) {
105      xQueueAddToSet(g_cmdQueue, g_ctrlSet);
106      xQueueAddToSet(g_statSem, g_ctrlSet);
107    }
108    g_events     = xEventGroupCreate();                               // [K9]
109    g_statTimer  = xTimerCreate("stat", pdMS_TO_TICKS(STAT_TIMER_MS), pdTRUE,
nullptr, stat_timer_cb);  // [K11]
110
111    if (!g_btn2Sem || !g_lightMutex || !g_serialMutex || !g_cmdQueue || !g_statSem ||
!g_ctrlSet ||
112        !g_events || !g_statTimer) {
113      Serial.println("[BOOT] LOI: khong du heap de tao doi tuong kernel");
114      for (;;) delay(1000);
```

`xTaskCreatePinnedToCore(hàm, tên, stack, tham số, priority, &handle, core)` là bản ESP-IDF của `xTaskCreate` có thêm lõi chạy. Lưu ý trên ESP-IDF **stack tính bằng byte** (FreeRTOS gốc tính bằng word). Handle được lưu vào mảng `g_task[]` để Task 7 điều khiển task lúc chạy.

src/main.cpp (dòng 122–131)

```
122    // Tạo task [K1] + truyền tham số [K2]
123    //           hàm            id   stack        tham số
priority          core
124    create_task(task1_led1,    ID_T1, STK_SMALL,   (void *)(uintptr_t)LED1_PERIOD_MS,
PRIO_T1_LED1,    CORE_IO);
125    create_task(task2_sensor,  ID_T2, STK_SENSOR,  nullptr,
PRIO_T2_SENSOR,  CORE_BG);
126    create_task(task3_btn1,    ID_T3, STK_SMALL,   (void *)&s_btn1Cfg,
PRIO_T3_BTN1,    CORE_IO);
127    create_task(task4_btn2,    ID_T4, STK_SMALL,   (void *)&s_btn2Cfg,
PRIO_T4_BTN2,    CORE_IO);
128    create_task(task5_led3,    ID_T5, STK_SMALL,   nullptr,
PRIO_T5_LED3,    CORE_IO);
129    create_task(task6_oled,    ID_T6, STK_OLED,    nullptr,
PRIO_T6_OLED,    CORE_BG);
130    create_task(task7_control, ID_T7, STK_CONTROL, nullptr,
PRIO_T7_CONTROL, CORE_BG);
131    create_task(task8_alarm,   ID_T8, STK_SMALL,   nullptr,
PRIO_T8_ALARM,   CORE_BG);
```

Truyền tham số qua pvParameters [K2]: Task 1 nhận chu kỳ (ép số vào con trỏ), Task 3/4 nhận con trỏ tới cấu hình nút. Biến cấu hình khai báo `static` vì task chạy mãi mãi, còn biến cục bộ của `setup()` sẽ mất khi `setup()` kết thúc.

src/main.cpp (dòng 57–58)

```
57  static const ButtonCfg_t s_btn1Cfg = {PIN_BTN1, "BTN1"};
58  static const ButtonCfg_t s_btn2Cfg = {PIN_BTN2, "BTN2"};
```

Sau `setup()`, `loop()` không còn việc nên tự xoá loopTask bằng `vTaskDelete(NULL)` [K14] để trả lại RAM stack của nó.

4.2. Task 1: LED1 nháy T = 200 ms với vTaskDelayUntil

src/task_io.cpp (dòng 10–22)

```
10  void task1_led1(void *pv) {
11    const uint32_t   periodMs = (uint32_t)(uintptr_t)pv;
12    const TickType_t half     = pdMS_TO_TICKS(periodMs / 2);
13    bool level = false;
14
15    app_sync_start(EV_T1);
16    TickType_t wake = xTaskGetTickCount();        // mốc thời gian tuyệt đối
17    for (;;) {
18      level = !level;
19      digitalWrite(PIN_LED1, level);
20      // Ngủ tới (wake + half): thời gian chạy thân vòng lặp không làm chu kỳ dài
thêm.
21      // Nếu dùng vTaskDelay(half) thì chu kỳ thực = half + thời gian chạy -> trôi
dần.
22      vTaskDelayUntil(&wake, half);
```

`vTaskDelay(n)` ngủ **n tick tính từ lúc gọi**, nên chu kỳ thực = n + thời gian chạy thân vòng lặp và bị trôi dần. `vTaskDelayUntil(&wake, n)` ngủ tới **mốc tuyệt đối** `wake + n` rồi tự cập nhật `wake`, nên chu kỳ luôn đúng 100 ms, kể cả khi task bị task ưu tiên cao hơn chen ngang. Mỗi 100 ms đảo LED một lần, cho chu kỳ đầy đủ 200 ms đúng đề.

4.3. Task 2: đọc cảm biến, lưu biến toàn cục, bảo vệ bằng mutex

Biến toàn cục theo đề là struct `g_light` gồm 5 trường. CPU không ghi 5 trường trong một lệnh, nên nếu Task 6 (ở **lõi khác**) đọc đúng lúc Task 2 đang ghi thì sẽ thấy bản ghi nửa cũ nửa mới. Mutex bảo đảm hai bên không vào cùng lúc:

src/task_sensor.cpp (dòng 39–51)

```
39      // Ghi biến toàn cục dưới mutex [K6]: Task 6 / Task 7 không bao giờ đọc được
40      // bản ghi "nửa cũ nửa mới" (5 trường, không thể ghi nguyên tử).
41      if (xSemaphoreTake(g_lightMutex, pdMS_TO_TICKS(20)) == pdTRUE) {
42        g_light.raw      = raw;
43        g_light.mv       = mv;
44        g_light.lightPct = pct;
45        g_light.count++;
46        g_light.tick     = xTaskGetTickCount();
47        xSemaphoreGive(g_lightMutex);
48      } else {
49        app_log("T2 khong lay duoc mutex, bo mau nay");
50      }
51      xEventGroupSetBits(g_events, EV_SENSOR_OK);    // [K9] báo Task 6: đã có dữ
liệu thật
```

Mutex khác binary semaphore ở chỗ có **kế thừa ưu tiên (priority inheritance)**: nếu task ưu tiên thấp đang giữ mutex mà task ưu tiên cao chờ, task thấp được tạm nâng ưu tiên để làm xong và nhả sớm, tránh **đảo ưu tiên**. Timeout 20 ms thay vì `portMAX_DELAY`: hết

giờ thì bỏ mẫu này và ghi log, không treo task. Giá trị đo là trung bình 8 mẫu ADC để giảm nhiễu; `vTaskDelayUntil` giữ nhịp đúng 500 ms.

4.4. Task 3: BTN1 đảo LED2, vòng lặp quét có chống dội

src/task_io.cpp (dòng 36–45)

```
36  static bool debounce_pressed(Debouncer_t *d) {
37    uint8_t now = (uint8_t)digitalRead(d->pin);
38    if (now == d->stable) {
39      d->cnt = 0;
40      return false;
41    }
42    if (++d->cnt < BTN_STABLE_COUNT) return false;
43    d->stable = now;
44    d->cnt = 0;
45    return now == LOW;           // cạnh xuống = nhấn
```

src/task_io.cpp (dòng 51–65)

```
51  void task3_btn1(void *pv) {
52    const ButtonCfg_t *cfg = (const ButtonCfg_t *)pv;     // [K2]
53    // Lấy mức thật lúc khởi động làm mức ổn định: nút đang bị giữ khi cấp điện không
bị tính là 1 lần nhấn
54    Debouncer_t db = {cfg->pin, (uint8_t)digitalRead(cfg->pin), 0};
55
56    app_sync_start(EV_T3);
57    for (;;) {
58      if (debounce_pressed(&db)) {
59        g_led2On = !g_led2On;
60        digitalWrite(PIN_LED2, g_led2On ? HIGH : LOW);
61        app_log("%s nhan -> LED2 %s", cfg->name, g_led2On ? "ON" : "OFF");
62      }
63      // [K4] BẮT BUỘC: không có vTaskDelay thì task prio 3 này chạy liên tục,
64      // Task 1 (prio 2) cùng core 1 sẽ không bao giờ được chạy -> LED1 đứng yên.
65      vTaskDelay(pdMS_TO_TICKS(BTN_SCAN_MS));
```

Nút cơ khí dội nhiều xung trong vài ms. Bộ chống dội chỉ chấp nhận mức mới khi đọc giống nhau **3 lần liên tiếp cách nhau 10 ms**, và chỉ báo ở cạnh nhấn, nên một lần nhấn đảo LED2 đúng một lần. Mức ổn định ban đầu lấy từ chân thật, để nút đang bị giữ lúc cấp điện không bị tính là một lần nhấn.

`vTaskDelay(10 ms)` là **bắt buộc**: Task 3 ưu tiên 3 > Task 1 ưu tiên 2 trên cùng core 1. Nếu bỏ dòng này, Task 3 không bao giờ Blocked và Task 1 bị đói, LED1 đứng yên. Đây là ví dụ trực tiếp của lập lịch ưu tiên có chiếm quyền ở mục 2.3.

4.5. Task 4 và Task 5: đồng bộ bằng binary semaphore

src/task_io.cpp (dòng 81–85)

```
81        // Binary semaphore chỉ có 0 hoặc 1. Nếu Task 5 đang bận (LED3 đang sáng) và
đã có
82        // 1 token chờ sẵn thì give trả pdFALSE: lần nhấn này "gộp" vào lần trước.
83        BaseType_t r = xSemaphoreGive(g_btn2Sem);
84        app_log("%s nhan #%lu -> give sem %s", cfg->name, (unsigned long)presses,
85                r == pdTRUE ? "OK" : "(da co token, bo qua)");
```

src/task_io.cpp (dòng 94–117)

```
 94  void task5_led3(void *pv) {
 95    app_sync_start(EV_T5);
 96    for (;;) {
 97      // Blocked vô thời hạn: task không tốn CPU cho tới khi Task 4 give [K5]
 98      xSemaphoreTake(g_btn2Sem, portMAX_DELAY);
 99
100      TickType_t offAt = xTaskGetTickCount() + pdMS_TO_TICKS(LED3_ON_MS);
101      // [K7] 3 trường của g_led3 phải đổi CÙNG LÚC: Task 6 ở core 0 có thể đọc xen
vào giữa.
102      // Spinlock + tắt ngắt ngắn hạn => Task 6 thấy hoặc trạng thái cũ, hoặc trạng
thái mới, không lẫn.
103      taskENTER_CRITICAL(&g_led3Lock);
104      g_led3.on    = true;
105      g_led3.offAt = offAt;
106      g_led3.timesOn++;
107      taskEXIT_CRITICAL(&g_led3Lock);
108      digitalWrite(PIN_LED3, HIGH);
109      app_log("T5 nhan sem -> LED3 ON 3s (lan %lu)", (unsigned long)g_led3.timesOn);
110
111      vTaskDelay(pdMS_TO_TICKS(LED3_ON_MS));     // [K4]
112
113      digitalWrite(PIN_LED3, LOW);
114      taskENTER_CRITICAL(&g_led3Lock);
115      g_led3.on = false;
116      taskEXIT_CRITICAL(&g_led3Lock);
117      app_log("T5 LED3 OFF, doi semaphore tiep");
```

Semaphore được tạo ở trạng thái rỗng. Task 5 gọi `xSemaphoreTake(..., portMAX_DELAY)` nên nằm ở Blocked, **không tốn CPU** cho tới khi Task 4 give. Khi give, Task 5 (ưu tiên 4, cao nhất) chiếm CPU ngay và bật LED3. Đây là đồng bộ "báo sự kiện" giữa hai task, đúng vai trò của binary semaphore.

Hành vi khi nhấn nhiều lần: semaphore chỉ giữ tối đa 1 token. Nhấn thêm trong lúc LED3 đang sáng thì lần đầu được giữ lại (LED3 sẽ sáng thêm 3 s nữa), các lần sau `xSemaphoreGive` trả `pdFALSE` và bị bỏ. Kết quả đo ở mục 5 xác nhận điều này. Nếu muốn mỗi lần nhấn đều được tính thì đổi sang **counting semaphore** (`xSemaphoreCreateCounting`), nhưng đề yêu cầu binary nên giữ binary.

Critical section [K7]: ba trường của `g_led3` phải đổi cùng lúc để Task 6 tính đếm ngược đúng. Đoạn này rất ngắn (vài lệnh gán), nên dùng `taskENTER_CRITICAL(&spinlock)` thay vì mutex. Trên ESP32 hai lõi, critical section phải có spinlock: tắt ngắt chỉ chặn lõi hiện tại, spinlock chặn lõi còn lại.

4.6. Task 6: hiển thị OLED mỗi 500 ms

src/task_display.cpp (dòng 45–65)

```
45      // --- 1. Chụp nhanh dữ liệu dùng chung (giữ khoá càng ngắn càng tốt) ---
46      LightData_t light = {};
47      if (xSemaphoreTake(g_lightMutex, pdMS_TO_TICKS(20)) == pdTRUE) {   // [K6]
48        light = g_light;                 // copy struct rồi nhả mutex ngay
49        xSemaphoreGive(g_lightMutex);
50      }
51      taskENTER_CRITICAL(&g_led3Lock);                                    // [K7]
52      Led3State_t led3 = g_led3;
53      taskEXIT_CRITICAL(&g_led3Lock);
54      bool led2 = g_led2On;
55      bool dark = g_dark;
56
57      // Thời gian LED3 còn sáng (ms). Phép trừ unsigned vẫn đúng khi tick tràn số.
58      TickType_t now = xTaskGetTickCount();
59      int32_t remainMs = 0;
60      if (led3.on) {
61        remainMs = (int32_t)(led3.offAt - now) * portTICK_PERIOD_MS;
62        if (remainMs < 0) remainMs = 0;
63      }
64
65      // --- 2. Vẽ (ngoài vùng khoá: I2C chậm ~25 ms, không giữ mutex lúc này) ---
```

Task 6 **chụp nhanh** dữ liệu dưới khoá (copy struct mất vài µs) rồi nhả ngay; việc vẽ và gửi I2C (~25 ms) làm ngoài vùng khoá, nên Task 2 không bao giờ phải chờ lâu. Thời gian còn lại của LED3 = `offAt − now`; phép trừ không dấu vẫn đúng khi bộ đếm tick tràn. Màn hình hiện: LED2 ON/OFF, % ánh sáng kèm mV và thanh mức, LED3 đếm ngược theo 0,1 s kèm thanh tiến trình, rồi "OFF" khi tắt.

Trước vòng lặp, Task 6 chờ bit `EV_SENSOR_OK` (tối đa 1 s) để không hiện số 0 giả lúc mới bật. Nếu không thấy OLED, task thử khởi tạo lại mỗi 5 s, các task khác vẫn chạy bình thường.

4.7. Các kỹ thuật mở rộng

a) Recursive mutex bảo vệ Serial [K6b]

src/task_control.cpp (dòng 25–39)

```
25  void app_log(const char *fmt, ...) {
26    char text[LOG_TEXT_LEN];
27    TickType_t tick = xTaskGetTickCount();
28    va_list ap;
29    va_start(ap, fmt);
30    vsnprintf(text, sizeof(text), fmt, ap);
31    va_end(ap);
32
33    if (xSemaphoreTakeRecursive(g_serialMutex, pdMS_TO_TICKS(LOG_WAIT_MS)) == pdTRUE)
{
34      Serial.printf("[%7lu] %s\n", (unsigned long)(tick * portTICK_PERIOD_MS), text);
35      xSemaphoreGiveRecursive(g_serialMutex);
36    } else {
37      taskENTER_CRITICAL(&s_dropLock);
38      g_logDropped = g_logDropped + 1;
39      taskEXIT_CRITICAL(&s_dropLock);
```

Serial là tài nguyên dùng chung: nếu hai task in cùng lúc, chữ bị lẫn vào nhau. Mỗi task tự in log, nhưng phải giữ `g_serialMutex` trong lúc in. Dùng **mutex** chứ không dùng critical section vì in UART mất hàng trăm µs, không được tắt ngắt lâu như vậy. Nhờ **kế thừa ưu tiên**, khi Task 5 (prio 4) chờ mutex mà Task 6 (prio 1) đang giữ, Task 6 được tạm nâng lên prio 4 để in xong và nhả sớm. Chờ tối đa 10 ms, quá thì bỏ dòng log và đếm vào `g_logDropped`, task gọi không bị treo.

Mutex là loại **recursive** vì lệnh `stat` của Task 7 giữ mutex suốt lúc in bảng nhiều dòng (để không task nào chen vào giữa), và bên trong vẫn gọi `app_log()`. Với mutex thường, task tự lấy lại mutex mình đang giữ sẽ tự khoá chết; recursive mutex đếm số lần lấy và chỉ mở khi nhả đủ số lần:

src/task_control.cpp (dòng 102–120)

```
102  static void print_stat() {
103    xSemaphoreTakeRecursive(g_serialMutex, portMAX_DELAY);
104    Serial.println("ID  Ten          Trang thai  Prio  Stack con(byte)  Core");
105    for (int i = 0; i < NUM_TASKS; i++) {
106      TaskHandle_t h = g_task[i];
107      if (!h) continue;
108      eTaskState st = (h == xTaskGetCurrentTaskHandle()) ? eRunning :
eTaskGetState(h);
109      BaseType_t core = -1;
110  #if ESP_IDF_VERSION_MAJOR >= 5
111      core = xTaskGetCoreID(h);
112  #endif
113      Serial.printf("%-3d %-12s %-11s %-5u %-16u %ld\n", i + 1, g_taskName[i],
state_str(st),
114                    (unsigned)uxTaskPriorityGet(h),
(unsigned)uxTaskGetStackHighWaterMark(h),
115                    (long)core);
116    }
117    app_log("heap %lu B, thap nhat %lu B, log bo %lu",     // gọi app_log khi ĐANG
giữ mutex: nhờ recursive
118            (unsigned long)esp_get_free_heap_size(),
119            (unsigned long)esp_get_minimum_free_heap_size(), (unsigned
long)g_logDropped);
120    xSemaphoreGiveRecursive(g_serialMutex);
```

c) Queue lệnh + queue set: Task 7 chờ hai nguồn cùng lúc [K8]

Task 7 nhận sự kiện từ hai nơi: dòng lệnh người dùng gõ ở Serial Monitor, và tín hiệu định kỳ của software timer. Callback UART (chạy mỗi khi có byte tới) ghép ký tự thành dòng rồi đẩy vào `g_cmdQueue`; queue copy dữ liệu theo giá trị nên callback dùng lại bộ đệm ngay được:

src/task_control.cpp (dòng 51–57)

```
 51    while (Serial.available()) {
 52      char ch = (char)Serial.read();
 53      if (ch == '\r') continue;
 54      if (ch == '\n') {
 55        if (len) {
 56          line.text[len] = 0;
 57          xQueueSend(g_cmdQueue, &line, 0);       // copy theo giá trị -> dùng lại
'line' ngay
```

Một task chỉ chờ được một đối tượng tại một thời điểm. **Queue set** gom `g_cmdQueue` và `g_statSem` lại; `xQueueSelectFromSet` cho Task 7 nằm Blocked tới khi **một trong hai** có dữ liệu, rồi trả về đúng đối tượng đó để lấy ra:

src/main.cpp (dòng 103–106)

```
103    g_ctrlSet    = xQueueCreateSet(CMD_QUEUE_LEN + 1);
104    if (g_ctrlSet) {
105      xQueueAddToSet(g_cmdQueue, g_ctrlSet);
106      xQueueAddToSet(g_statSem, g_ctrlSet);
```

src/task_control.cpp (dòng 177–188)

```
177      QueueSetMemberHandle_t who = xQueueSelectFromSet(g_ctrlSet, portMAX_DELAY);
178
179      if (who == (QueueSetMemberHandle_t)g_cmdQueue) {
180        CmdLine_t cmd;
181        xQueueReceive(g_cmdQueue, &cmd, 0);        // chắc chắn có, không cần chờ
182        handle_cmd(cmd.text);
183      } else if (who == (QueueSetMemberHandle_t)g_statSem) {
184        xSemaphoreTake(g_statSem, 0);
185        uint32_t upS = (uint32_t)(xTaskGetTickCount() * portTICK_PERIOD_MS / 1000);
186        app_log("[STAT] up %lus heap %lu min %lu drop %lu", (unsigned long)upS,
187                (unsigned long)esp_get_free_heap_size(),
188                (unsigned long)esp_get_minimum_free_heap_size(), (unsigned
long)g_logDropped);
```

Kích thước set phải bằng tổng số phần tử các thành viên (4 dòng lệnh + 1 token), và thành viên phải đang rỗng khi thêm vào. Nhờ vậy Task 7 không phải quét Serial định kỳ: khi không có lệnh và chưa tới giờ in thống kê, nó không tốn CPU.

d) Event group: điểm hẹn khởi động [K9]

src/main.cpp (dòng 63–64)

```
 63  void app_sync_start(EventBits_t myBit) {
 64    xEventGroupSync(g_events, myBit, EV_ALL_TASKS, portMAX_DELAY);
```

Mỗi task gọi `app_sync_start(bit của mình)` ở đầu. `xEventGroupSync` bật bit đó rồi chờ đủ 8 bit; khi task cuối cùng tới, cả 8 task cùng được thả ra. Nhờ vậy không task nào chạy khi task khác chưa tạo xong (ví dụ Task 2 gửi notification cho Task 8 khi handle của Task 8 chưa có).

e) Task notification làm mailbox một giá trị [K10]

src/task_sensor.cpp (dòng 54–59)

```
 54      bool newDark = dark ? (pct < DARK_OFF_PCT) : (pct < DARK_ON_PCT);
 55      if (newDark != dark) {
 56        dark = newDark;
 57        // [K10] Task notification thay cho queue 1 phần tử: nhanh hơn, không cần tạo
đối tượng.
 58        // Giá trị gửi kèm: bit 8 = trạng thái tối, bit 0..7 = % ánh sáng.
 59        xTaskNotify(g_task[ID_T8], ((uint32_t)dark << 8) | pct,
eSetValueWithOverwrite);
```

src/task_sensor.cpp (dòng 74–76)

```
 74      xTaskNotifyWait(0, ULONG_MAX, &v, portMAX_DELAY);
 75      bool dark = (v >> 8) & 1;
 76      g_dark = dark;
```

Task 2 chỉ báo khi trạng thái tối **đổi** (có trễ: vào tối khi < 25 %, ra khi > 35 %), gửi kèm giá trị 32 bit ghép cờ tối và % ánh sáng. `eSetValueWithOverwrite` ghi đè giá trị cũ, đúng ý nghĩa "chỉ cần trạng thái mới nhất". So với queue 1 phần tử, notification không cần tạo đối tượng và nhanh hơn, nhưng chỉ gửi được cho đúng một task.

f) Software timer [K11]

src/task_control.cpp (dòng 74–75)

```
 74  void stat_timer_cb(TimerHandle_t t) {
 75    xSemaphoreGive(g_statSem);
```

Timer tạo bằng `xTimerCreate(..., pdTRUE /*auto-reload*/, ...)`, 10 s gọi callback một lần. Callback chạy trong **timer daemon task** của FreeRTOS, không phải trong ngắt, nhưng **không được block** vì sẽ làm trễ mọi timer khác. Vì vậy callback chỉ give `g_statSem` để đánh thức Task 7 qua queue set; việc in thống kê (chậm, phải chờ mutex Serial) để Task 7 làm.

g) Điều khiển và giám sát task lúc chạy [K12][K13]

src/task_control.cpp (dòng 148–160)

```
148      if (c == 's') {
149        if (a - 1 == ID_T7) { app_log("Khong suspend chinh Task 7 (se mat lenh)");
return; }
150        vTaskSuspend(h);
151        app_log("Da suspend %s", g_taskName[a - 1]);
152        return;
153      }
154      if (c == 'r') {
155        vTaskResume(h);
156        app_log("Da resume %s", g_taskName[a - 1]);
157        return;
158      }
159      if (c == 'p' && n == 3 && b >= 0 && b < configMAX_PRIORITIES - 1) {
160        vTaskPrioritySet(h, b);
```

Task 7 xử lý lệnh từ Serial Monitor: `s n` / `r n` gọi `vTaskSuspend` / `vTaskResume` (ví dụ `s 1` làm LED1 ngừng nháy, `r 1` cho chạy lại), `p n x` gọi `vTaskPrioritySet`, `give` cấp semaphore thay cho BTN2, `stat` in trạng thái từng task (`eTaskGetState`), priority, core và **stack còn dư nhỏ nhất** (`uxTaskGetStackHighWaterMark`), cùng heap còn trống. Không cho suspend chính Task 7 vì sẽ không còn ai nhận lệnh.

Bảng 5. Ý nghĩa các hàm FreeRTOS đã dùng

Hàm	Ý nghĩa
`xTaskCreatePinnedToCore`	Tạo task, ghim vào 1 lõi (ESP-IDF)
`vTaskDelay`	Ngủ n tick tính từ lúc gọi (Blocked)
`vTaskDelayUntil`	Ngủ tới mốc tuyệt đối, giữ chu kỳ cố định
`vTaskDelete(NULL)`	Xoá chính task đang chạy
`vTaskSuspend / vTaskResume`	Treo / cho chạy lại task khác
`vTaskPrioritySet / uxTaskPriorityGet`	Đổi / đọc ưu tiên lúc chạy
`uxTaskGetStackHighWaterMark`	Stack còn dư nhỏ nhất từ khi task chạy

Hàm	Ý nghĩa
`xSemaphoreCreateBinary / Give / Take`	Tạo, cấp, chờ binary semaphore
`xSemaphoreCreateMutex`	Tạo mutex (có kế thừa ưu tiên)
`xQueueCreate / xQueueSend / xQueueReceive`	Tạo, gửi, nhận queue (copy theo giá trị)
`xQueueCreateSet / xQueueAddToSet / xQueueSelectFromSet`	Tạo queue set, thêm thành viên, chờ thành viên nào có dữ liệu
`xSemaphoreCreateRecursiveMutex / TakeRecursive / GiveRecursive`	Mutex lấy lại được nhiều lần trong cùng task
`xEventGroupSync / SetBits / WaitBits`	Điểm hẹn, bật cờ, chờ cờ
`xTaskNotify / xTaskNotifyWait`	Gửi / chờ notification kèm giá trị
`xTimerCreate / xTimerStart`	Tạo, khởi động software timer
`taskENTER_CRITICAL / taskEXIT_CRITICAL`	Vào / ra vùng găng (spinlock 2 lõi)
`pdMS_TO_TICKS`, `portMAX_DELAY`	Đổi ms sang tick; chờ vô hạn

5. Kiểm thử và kết quả

5.1. Build

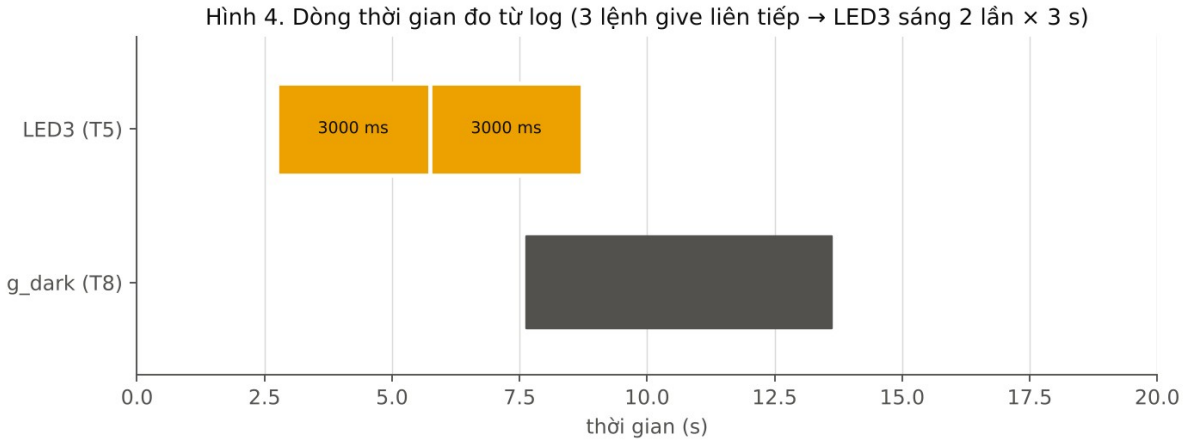
Build thành công cả hai env, không có cảnh báo nào trong code của project (bật `-Wall`):

Env	RAM	Flash
esp32s3	22 544 B / 320 KB (6,9 %)	414 451 B / 3,2 MB (12,4 %)
esp32dev	22 520 B / 320 KB (6,9 %)	381 171 B / 1,25 MB (29,1 %)

5.2. Chạy giả lập trên QEMU

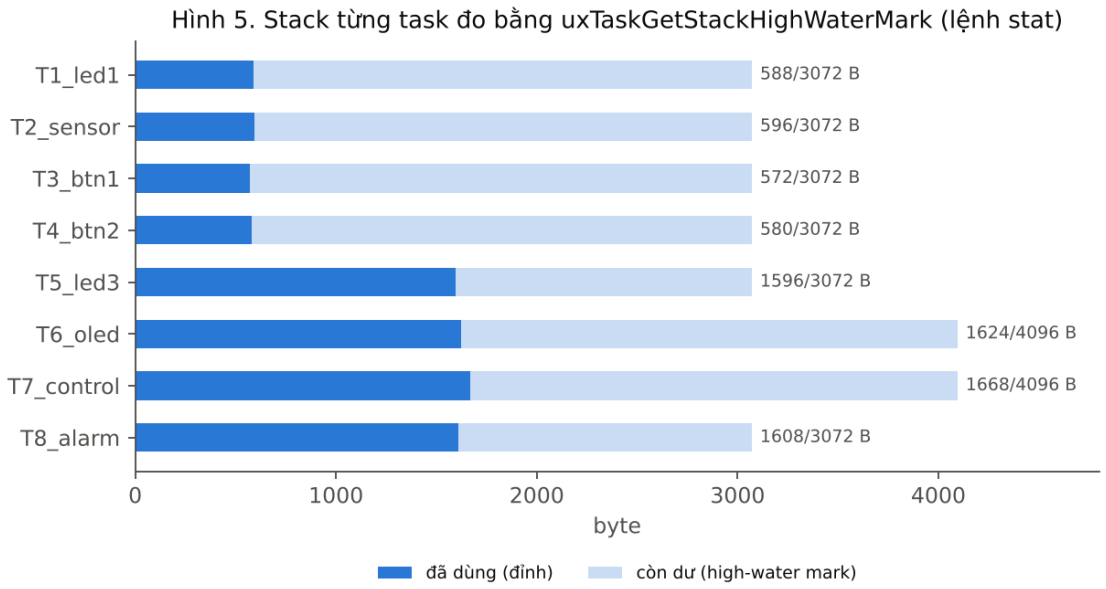
Ngoài kit thật, chương trình được chạy trên **QEMU của Espressif** (máy `esp32`, env `qemu_esp32`) để kiểm tra phần RTOS mà không cần phần cứng. QEMU không giả lập ADC nên env này dùng tín hiệu ánh sáng giả dạng tam giác chu kỳ 20 s (`SIM_NO_ADC`); mọi phần còn lại giống hệt bản nạp kit. Lệnh được gõ qua Serial: `help`, `stat`, ba lần `give` liên tiếp, `light`, `s 1`, `stat`, `r 1`, `p 6 2`, `s 7`, `xyz`, `stat`. Log đầy đủ: `docs/qemu_log.txt`. Trích đoạn:

```
[    628] T7 8 task da qua diem hen (xEventGroupSync), bat dau chay
[   2737] give sem tu lenh: OK
[   2978] give sem tu lenh: OK
[   3222] give sem tu lenh: da co token
[   2737] T5 nhan sem -> LED3 ON 3s (lan 1)
[   5737] T5 LED3 OFF, doi semaphore tiep
[   5737] T5 nhan sem -> LED3 ON 3s (lan 2)
[   8737] T5 LED3 OFF, doi semaphore tiep
[   7628] T8 CANH BAO: TOI (anh sang 22%)
[  13628] T8 het toi, binh thuong (anh sang 37%)
[  10627] [STAT] up 10s heap 239624 min 231444 drop 0
[  11268] Da suspend T1_led1
[  12883] Da resume T1_led1
[  13692] T6_oled prio -> 2
[  14504] Khong suspend chinh Task 7 (se mat lenh)
[  14905] Lenh khong hop le: 'xyz' (go help)
[  20627] [STAT] up 20s heap 239624 min 231444 drop 0
```



Hình 4. Dòng thời gian LED3 và cờ tối dựng từ log

Nhận xét: lần give thứ nhất làm LED3 sáng; lần thứ hai được giữ làm token; lần thứ ba báo "da co token" và bị bỏ. LED3 sáng **2 lần, mỗi lần đúng 3000 ms** (đo từ tick trong log), lần sau nối ngay lần trước, khớp phân tích ở 4.5. Task 8 báo tối khi ánh sáng xuống 22 % và hết tối khi lên 37 %, khớp ngưỡng 25/35 %. Thống kê in đúng mỗi 10 s (lúc 10,6 s và 20,6 s) qua đường timer → semaphore → queue set → Task 7, xen kẽ với lệnh người dùng mà không lẫn chữ. Không có log nào bị bỏ (`drop 0`).



Hình 5. Stack từng task

Lỗi tìm được nhờ giám sát stack: bản đầu cấp 2048 B cho các task nhỏ; lệnh `stat` cho thấy Task 5 và Task 8 chỉ còn ~650 B dư, vì `app_log()` gọi `vsnprintf` tốn nhiều stack. Đã tăng lên 3072 B, nay còn dư ≥ 1,4 KB. Lỗi thứ hai tìm được trên QEMU: nút đang ở mức thấp lúc khởi động bị tính là một lần nhấn; đã sửa bằng cách lấy mức thật của chân làm mức ổn định ban đầu (mục 4.4).

5.3. Kiểm thử trên kit

Bảng 6. Kịch bản kiểm thử trên kit (cột Kết quả điền khi chạy thật)

#	Thao tác	Mong đợi	Kết quả
1	Cấp điện, mở Serial Monitor	LED1 nháy 5 lần/s; dòng [T7] điểm hẹn	
2	Nhấn BTN1 vài lần	LED2 đảo mỗi lần nhấn, OLED cập nhật ON/OFF	
3	Nhấn BTN2 một lần	LED3 sáng 3 s, OLED đếm 3.0 → 0.0 rồi OFF	
4	Nhấn BTN2 3 lần nhanh	LED3 sáng 2 lượt × 3 s; log "da co token"	
5	Che / chiếu sáng quang trở	% ánh sáng đổi; dưới 25 % OLED hiện [TOI]	
6	Gõ `s 1`, rồi `r 1`	LED1 dừng, rồi nháy lại	
7	Gõ `stat` sau 10 phút	Mọi task còn stack dư, log bỏ = 0	

6. Kết luận

Chương trình hiện thực đủ 6 task của đề và áp dụng thêm các kỹ thuật FreeRTOS: tạo task ghim lõi có tham số, `vTaskDelayUntil`, binary semaphore, mutex, recursive mutex, critical section 2 lõi, queue kết hợp queue set, event group làm điểm hẹn, task

notification, software timer, điều khiển và giám sát task lúc chạy. Mỗi kỹ thuật có minh chứng code và được kiểm chứng qua build, chạy giả lập QEMU và đo log.

Bài học chính: task đúng chuẩn RTOS phải dành phần lớn thời gian ở trạng thái Blocked; mỗi tài nguyên dùng chung cần một cách bảo vệ phù hợp với độ dài đoạn găng (critical section cho vài lệnh, mutex cho đoạn dài hơn như in UART); và kích thước stack phải được đo chứ không đoán.

Hướng phát triển: chuyển nút nhấn sang ngắt GPIO kết hợp `xSemaphoreGiveFromISR`; ghi dữ liệu cảm biến vào stream buffer để vẽ đồ thị; đo tải CPU từng task bằng run-time stats.

Tài liệu tham khảo

[1] R. Barry, Mastering the FreeRTOS Real Time Kernel – A Hands-On Tutorial Guide, Real Time Engineers Ltd.

[2] FreeRTOS Kernel Documentation, https://www.freertos.org/Documentation

[3] Espressif Systems, ESP-IDF Programming Guide – FreeRTOS (SMP), https://docs.espressif.com/projects/esp-idf

[4] Espressif Systems, ESP32-S3 Technical Reference Manual.

[5] Espressif Systems, QEMU for ESP32, https://github.com/espressif/qemu

[6] Adafruit, SSD1306 OLED library, https://github.com/adafruit/Adafruit_SSD1306